App Health Tracker Deployment Report

Rajendra Guttedar

May 13, 2026

1 Overview

This report documents the end-to-end deployment of the App Health Tracker using Docker, Kubernetes (local and AWS EKS), and Terraform.

2 Stack

- App: Node.js + Express
- Container: Docker
- Local K8s: Minikube
- Cloud: AWS EKS + ECR
- IaC: Terraform

3 Docker (Local)

3.1 Steps

1. Build the Docker image.
2. Run the container locally.
3. Verify the app endpoints.

3.2 Commands

```
docker build -t app-health-tracker:1.0.0 .
docker run -d -p 3000:3000 --name app-health-tracker \
  -e APP_VERSION=1.0.0 \
  -e ENVIRONMENT=local \
  -e DEPLOYED_AT=local \
  app-health-tracker:1.0.0
curl http://localhost:3000/
curl http://localhost:3000/health
```

3.3 Screenshots

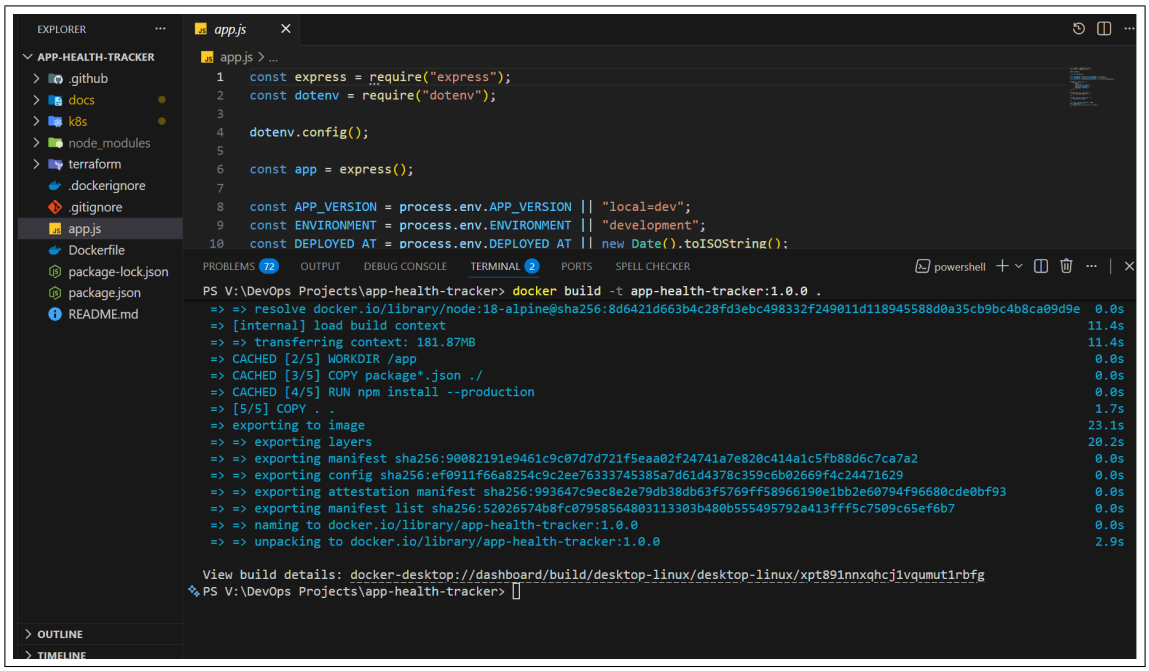


Figure 1: Docker build

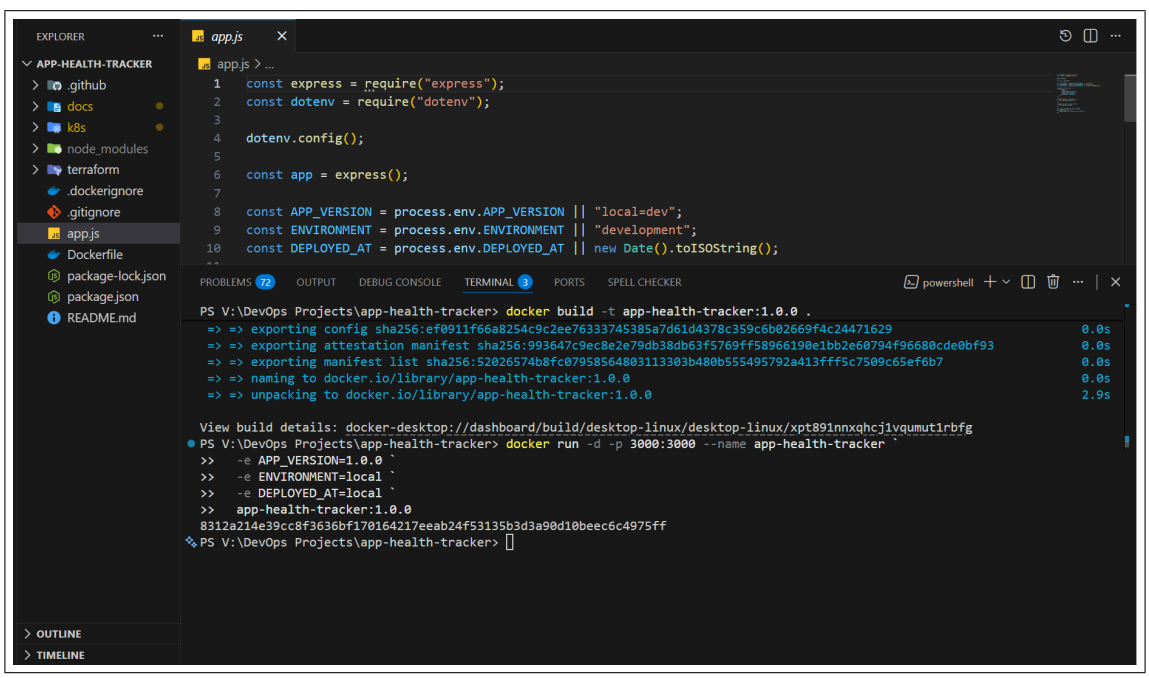


Figure 2: Docker run

```
PROBLEMS 72   OUTPUT   DEBUG CONSOLE   TERMINAL 4   PORTS   SPELL CHECKER                                  powershell

PS V:\DevOps Projects\app-health-tracker> docker build -t app-health-tracker:1.0.0 .
 => => exporting manifest list sha256:52026574b8fc0795856480311330 3b480b555495792a413fff5c7509c65ef6b7          0.0s
 => => naming to docker.io/library/app-health-tracker:1.0.0                                                      0.0s
 => => unpacking to docker.io/library/app-health-tracker:1.0.0                                                   2.9s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/xpt891nnxqhcj1vqumut1rbfg
PS V:\DevOps Projects\app-health-tracker> docker run -d -p 3000:3000 --name app-health-tracker `
>>   -e APP_VERSION=1.0.0 `
>>   -e ENVIRONMENT=local `
>>   -e DEPLOYED_AT=local `
>>   app-health-tracker:1.0.0
8312a214e39cc8f3636bf170164217eeab24f53135b3d3a90d10beec6c4975ff
PS V:\DevOps Projects\app-health-tracker> docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED          STATUS          PORTS
                    NAMES
8312a214e39c   app-health-tracker:1.0.0   "docker-entrypoint.s…"   35 seconds ago   Up 34 seconds   0.0.0.0:3000->3000/tcp, [:
:]:3000->3000/tcp   app-health-tracker
PS V:\DevOps Projects\app-health-tracker>
```

Figure 3: Docker ps

```
PROBLEMS 72   OUTPUT   DEBUG CONSOLE   TERMINAL 5   PORTS   SPELL CHECKER                                  powershell

PS V:\DevOps Projects\app-health-tracker> curl http://localhost:3000/

StatusCode        : 200
StatusDescription : OK
Content           : {"status":"UP","environment":"local","version":"1.0.0","deployedAt":"local"}
RawContent        : HTTP/1.1 200 OK
                    Connection: keep-alive
                    Keep-Alive: timeout=5
                    Content-Length: 76
                    Content-Type: application/json; charset=utf-8
                    Date: Wed, 13 May 2026 02:39:37 GMT
                    ETag: W/"4c-FBozgwIdkoau+Yp1z1ft...
Forms             : {}
Headers           : {[Connection, keep-alive], [Keep-Alive, timeout=5], [Content-Length, 76], [Content-Type,
                    application/json; charset=utf-8]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 76
```

Figure 4: Local app response (/) - terminal

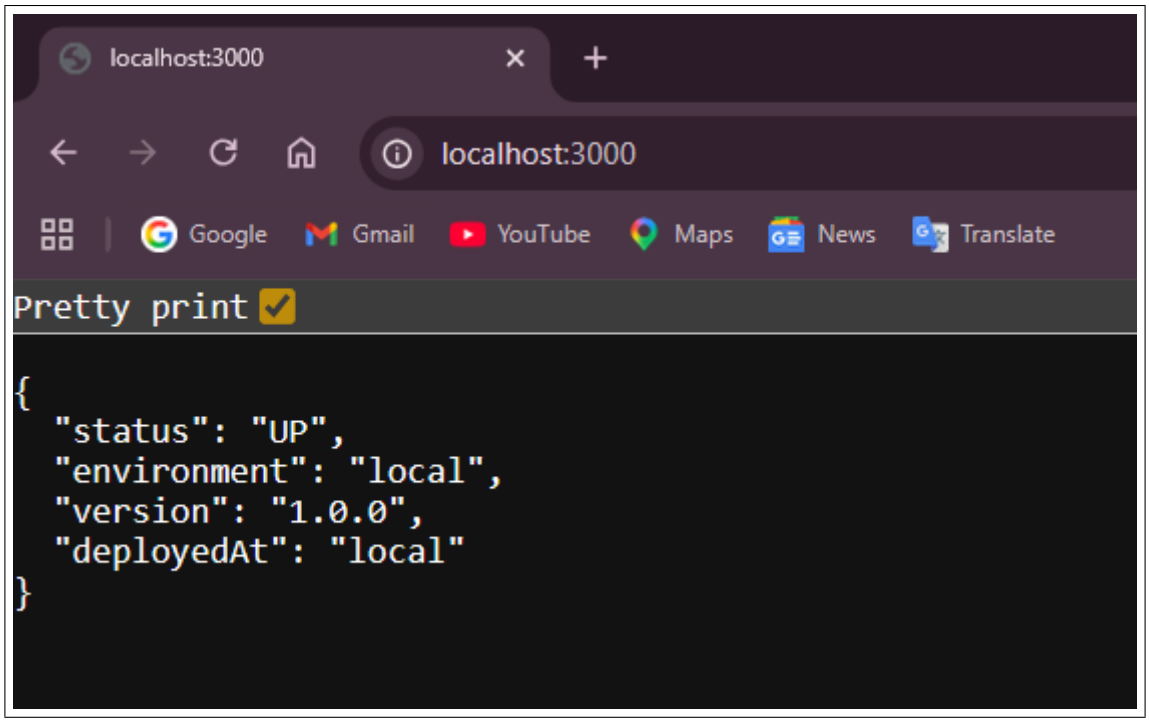


Figure 5: Local app response (/) - web

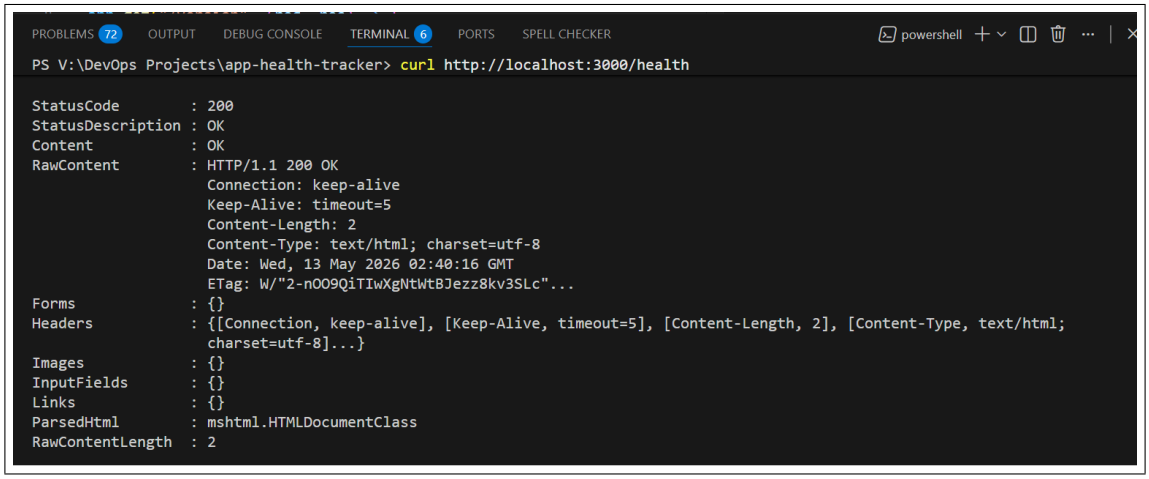


Figure 6: Local app response (/health) - terminal

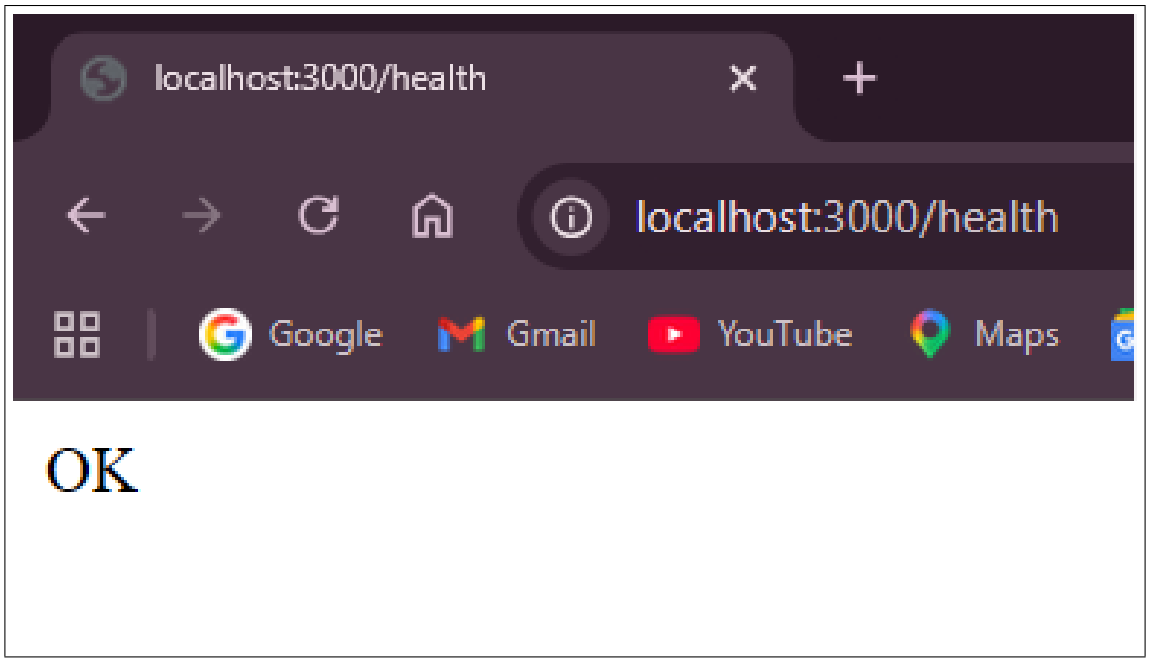


Figure 7: Local app response (/health) - web

4 Local Kubernetes (Minikube)

4.1 Steps

1. Start Minikube and configure Docker environment.
2. Build the Docker image locally.
3. Apply the local Kubernetes manifests.
4. Access the service URL.

4.2 Commands

```
minikube start
minikube docker-env --shell powershell | Invoke-Expression
docker build -t app-health-tracker:1.0.0 .
kubectl apply -f k8s\local\configmap.yaml
kubectl apply -f k8s\local\deployment.yaml
kubectl apply -f k8s\local\service.yaml
minikube service app-health-tracker --url
```

4.3 Screenshots

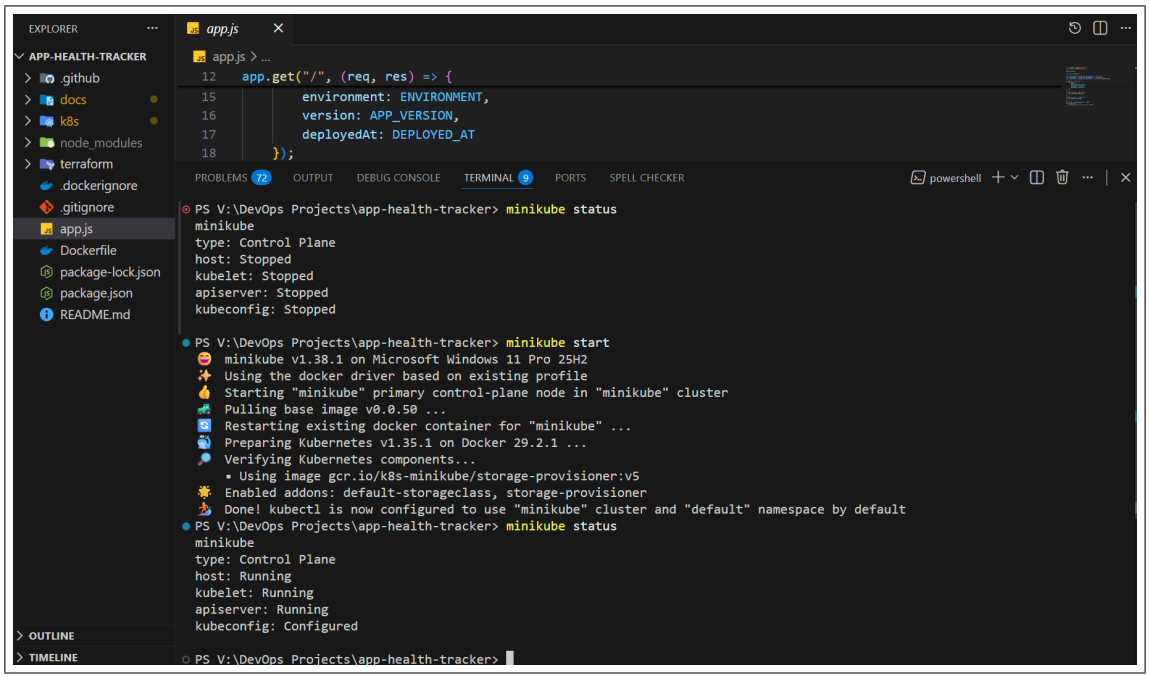


Figure 8: Minikube status

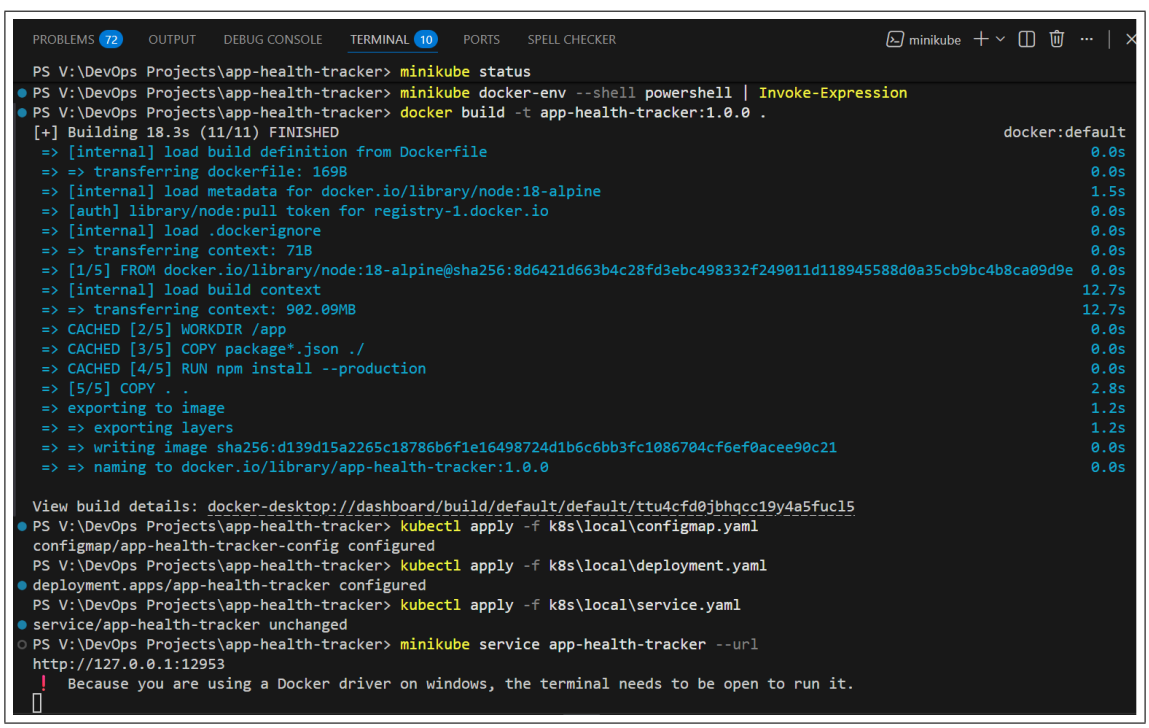


Figure 9: Local service URL

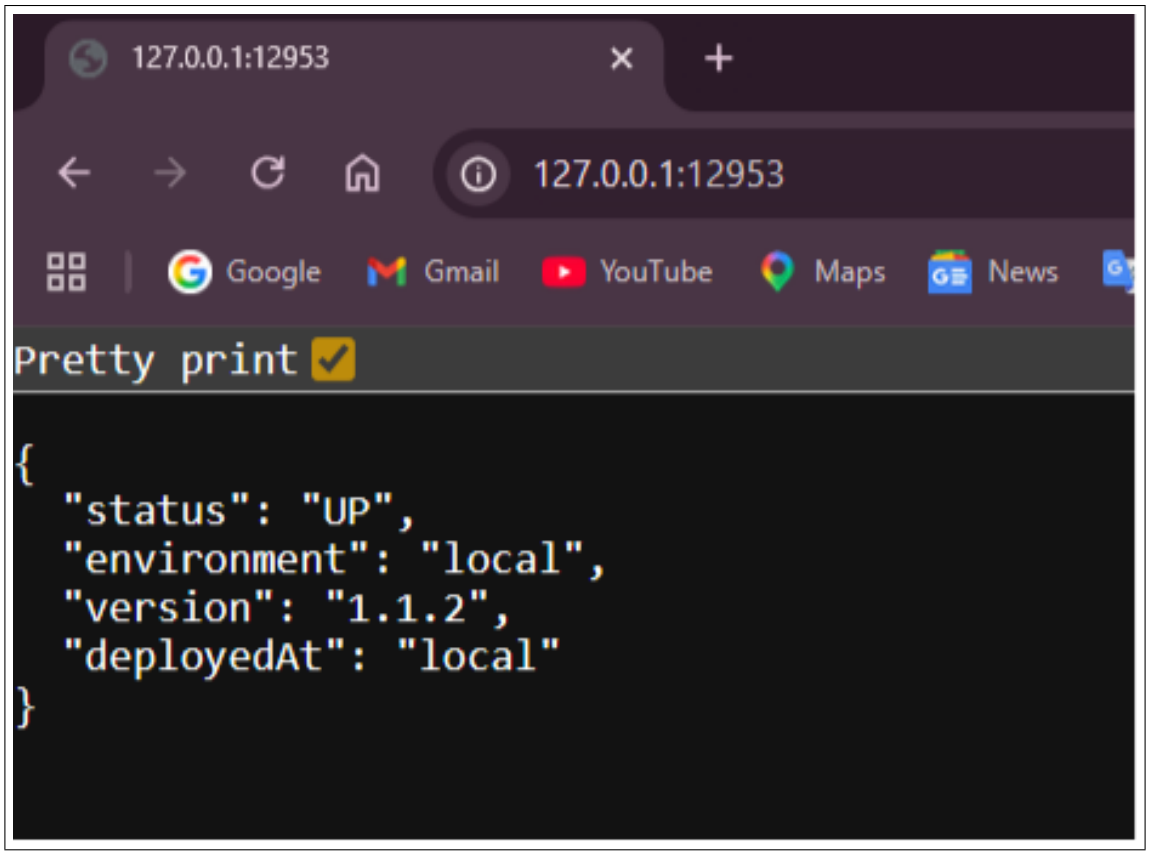


Figure 10: Local app response (/)

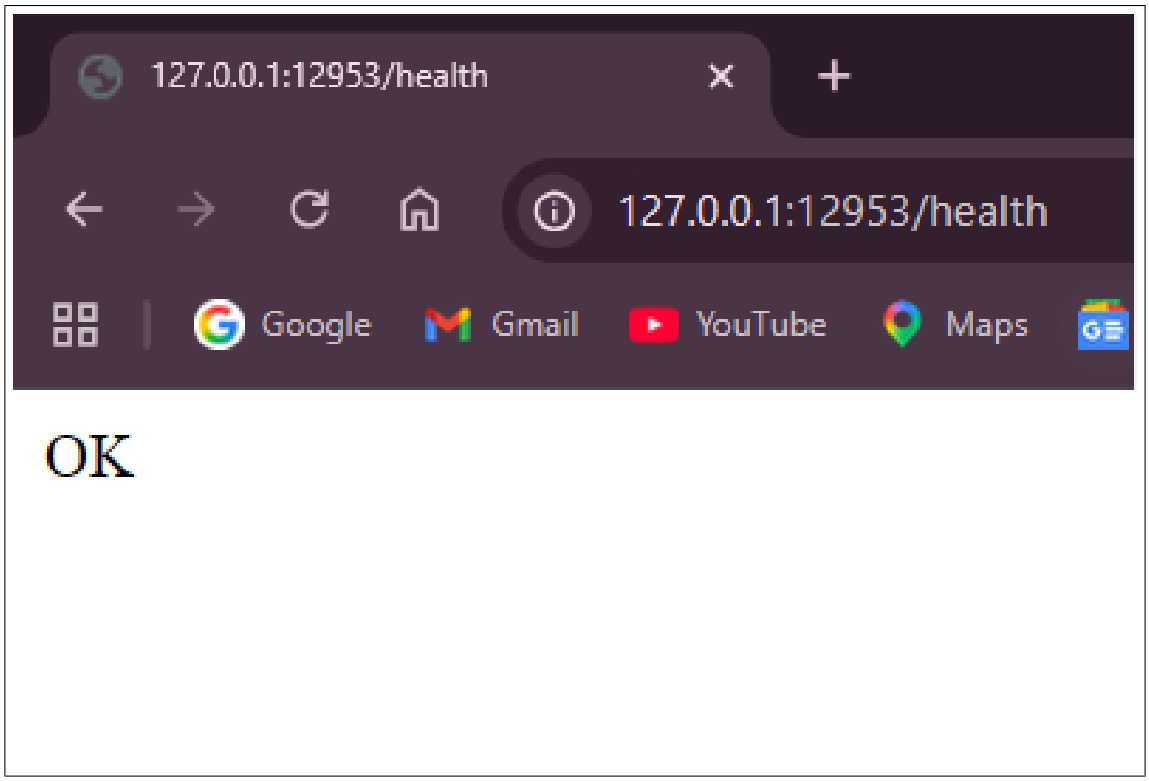


Figure 11: Local app response (/health)

5 Terraform (AWS EKS)

5.1 Steps

1. Initialize Terraform.
2. Plan and apply infrastructure changes.
3. Record outputs.

5.2 Commands

```
cd terraform
terraform init
terraform plan -out=tfplan
terraform apply tfplan
terraform output
```

5.3 Screenshots

```
PROBLEMS 72   OUTPUT   DEBUG CONSOLE   TERMINAL 13   PORTS   SPELL CHECKER

PS V:\DevOps Projects\app-health-tracker\terraform> terraform plan -out=tfplan

Changes to Outputs:
  + eks_cluster_ca_data  = (sensitive value)
  + eks_cluster_endpoint = (known after apply)
  + eks_cluster_name     = "app-health-tracker-eks"
  + eks_node_group_name  = "app-health-tracker-eks-ng"
  + private_subnet_ids   = [
      + (known after apply),
      + (known after apply),
    ]
  + public_subnet_ids    = [
      + (known after apply),
      + (known after apply),
    ]
  + vpc_id               = (known after apply)


Saved the plan to: tfplan

To perform exactly these actions, run the following command to apply:
    terraform apply "tfplan"
PS V:\DevOps Projects\app-health-tracker\terraform>
```

Figure 12: Terraform plan

```
PROBLEMS 72   OUTPUT   DEBUG CONSOLE   TERMINAL 14   PORTS   SPELL CHECKER   powershell - terraform

PS V:\DevOps Projects\app-health-tracker\terraform> terraform apply tfplan
aws_eks_cluster.this: Still creating... [06m30s elapsed]
aws_eks_cluster.this: Still creating... [06m40s elapsed]
aws_eks_cluster.this: Still creating... [06m50s elapsed]
aws_eks_cluster.this: Still creating... [07m00s elapsed]
aws_eks_cluster.this: Still creating... [07m10s elapsed]
aws_eks_cluster.this: Creation complete after 7m13s [id=app-health-tracker-eks]
aws_eks_node_group.default: Creating...
aws_eks_node_group.default: Still creating... [00m10s elapsed]
aws_eks_node_group.default: Still creating... [00m20s elapsed]
aws_eks_node_group.default: Still creating... [00m30s elapsed]
aws_eks_node_group.default: Still creating... [00m40s elapsed]
aws_eks_node_group.default: Still creating... [00m50s elapsed]
aws_eks_node_group.default: Still creating... [01m00s elapsed]
aws_eks_node_group.default: Still creating... [01m10s elapsed]
aws_eks_node_group.default: Still creating... [01m20s elapsed]
aws_eks_node_group.default: Still creating... [01m30s elapsed]
aws_eks_node_group.default: Still creating... [01m40s elapsed]
aws_eks_node_group.default: Creation complete after 1m50s [id=app-health-tracker-eks:app-health-tracker-eks-ng]

Apply complete! Resources: 30 added, 0 changed, 0 destroyed.
```

Figure 13: Terraform apply complete

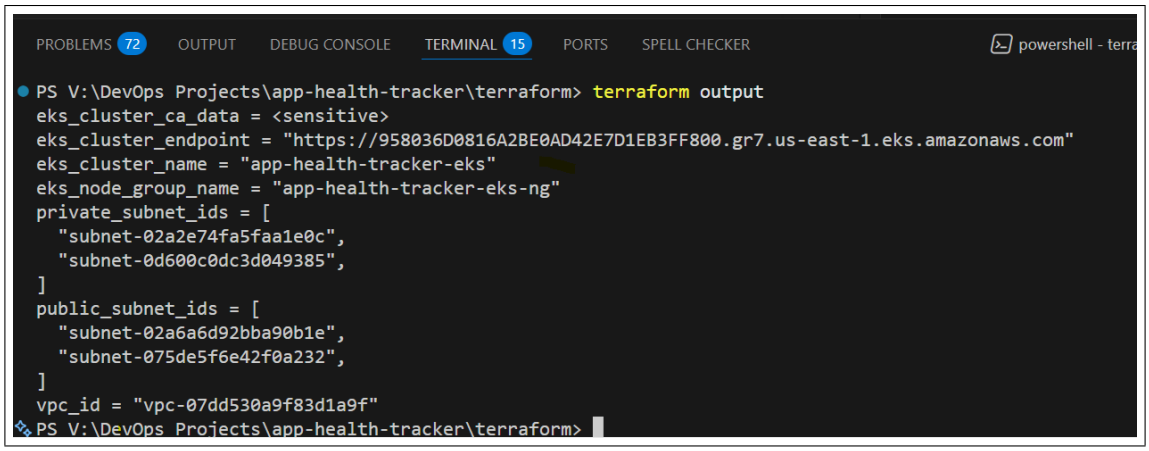


Figure 14: Terraform outputs

6 ECR Image Push

6.1 Steps

1. Build Docker image locally.
2. Create ECR repository.
3. Authenticate and push image to ECR.

6.2 Commands

```
$ACCOUNT = (aws sts get-caller-identity --query Account --output text)
$REGION = "us-east-1"
$REPO = "app-health-tracker"
$TAG = "1.0.0"
$URI = "$ACCOUNT.dkr.ecr.$REGION.amazonaws.com/${REPO}:${TAG}"

docker build -t app-health-tracker:1.0.0 .
aws ecr create-repository --repository-name $REPO --region $REGION
aws ecr get-login-password --region $REGION | docker login --username AWS --password-stdin "$A
docker tag app-health-tracker:1.0.0 $URI
docker push $URI
```

6.3 Screenshots

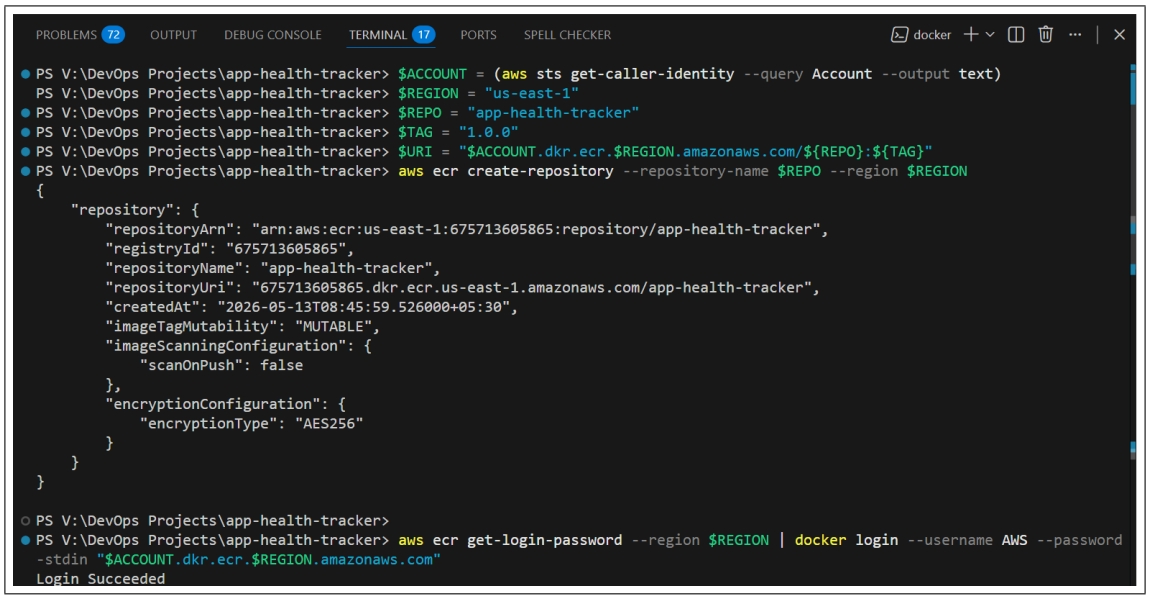


Figure 15: ECR repository created

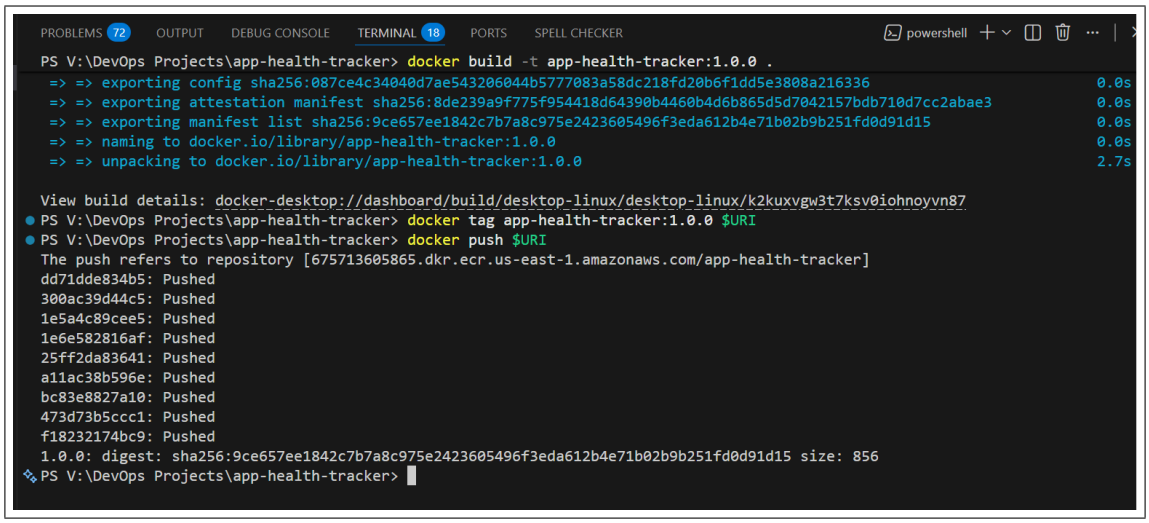


Figure 16: Docker push to ECR

7 Kubernetes (Production - AWS EKS)

7.1 Steps

1. Update kubeconfig to use EKS.
2. Apply production manifests.
3. Verify rollout and service.

7.2 Commands

```
aws eks update-kubeconfig --name app-health-tracker-eks --region us-east-1
kubectl get nodes

kubectl apply -f k8s\prod\configmap.yaml
kubectl apply -f k8s\prod\deployment.yaml
kubectl apply -f k8s\prod\service.yaml
kubectl rollout status deployment app-health-tracker
kubectl get svc app-health-tracker
```

7.3 Screenshots

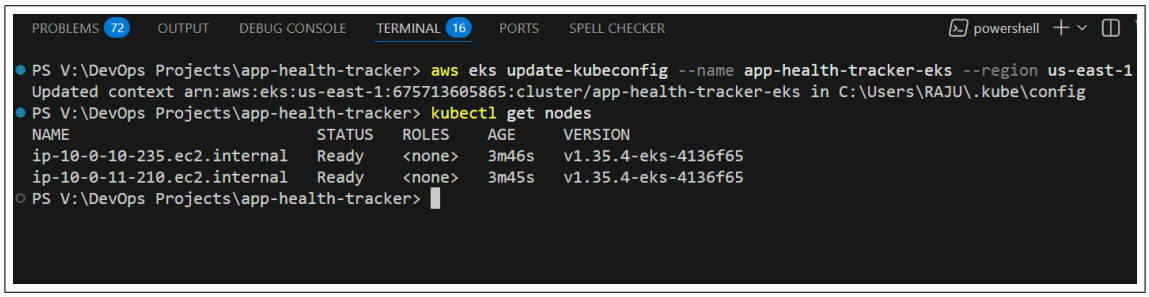


Figure 17: EKS nodes

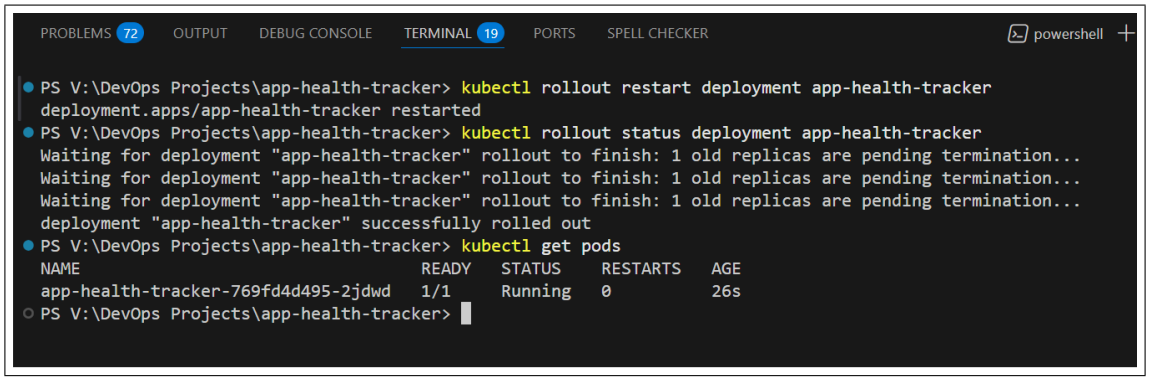


Figure 18: Deployment rollout

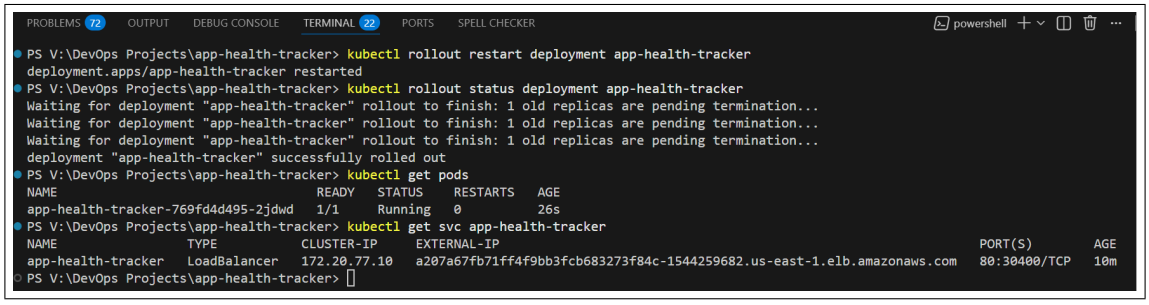


Figure 19: Service external IP

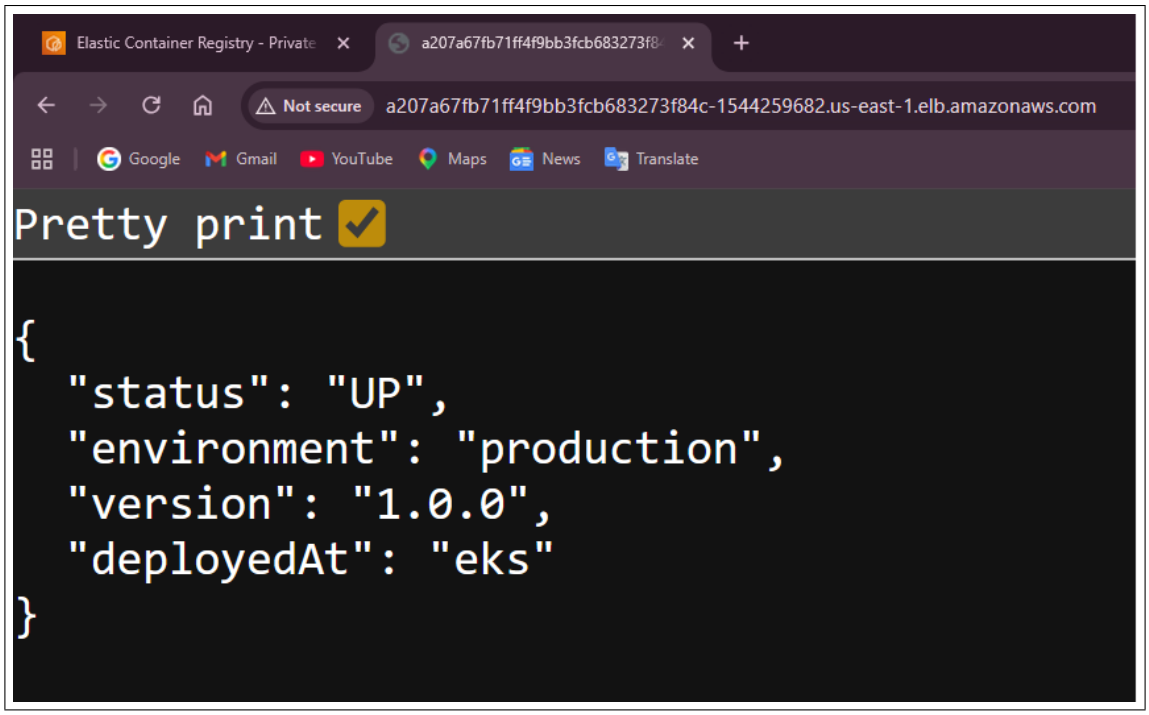


Figure 20: Production app response (/)

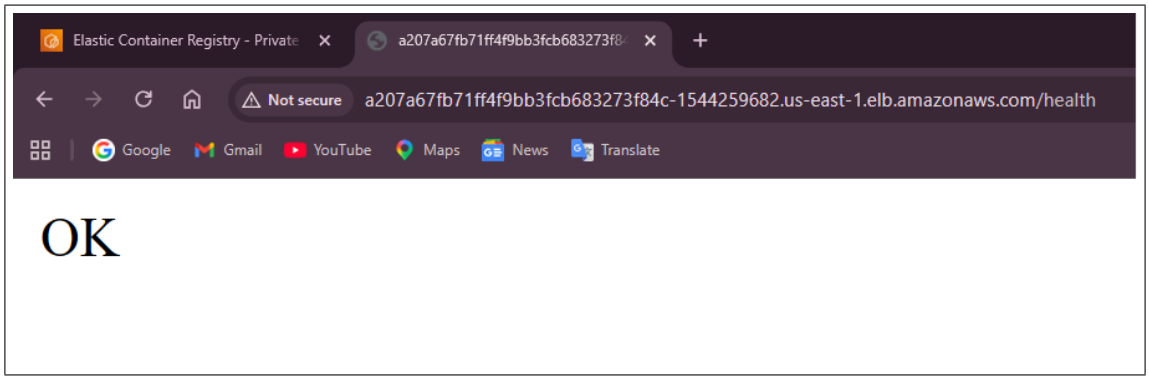


Figure 21: Production app response (/health)

8 EC2 Docker Deployment (GitHub Actions)

8.1 Steps

1. Push to main to trigger the workflow.
2. GitHub Actions connects to EC2 via SSH.
3. EC2 pulls the repo, builds the image, and runs the container.
4. Verify the service on EC2.

8.2 Commands (Manual EC2 SSH)

```
ssh -i <PATH_TO_EC2_SSH_KEY> ec2-user@<EC2_PUBLIC_IP>

# On EC2
sudo yum update -y
sudo yum install git docker -y
sudo service docker start
sudo usermod -aG docker ec2-user

git clone https://github.com/Rajendra0309/app-health-deployment-tracker.git
cd app-health-deployment-tracker
docker build -t app-health-tracker .
docker run -d --name app-health-tracker -p 3000:3000 \
  -e APP_VERSION=manual \
  -e ENVIRONMENT=production \
  -e DEPLOYED_AT=$(date -u +"%Y-%m-%dT%H:%M:%SZ") \
  app-health-tracker
curl http://localhost:3000/health
```

8.3 Screenshots

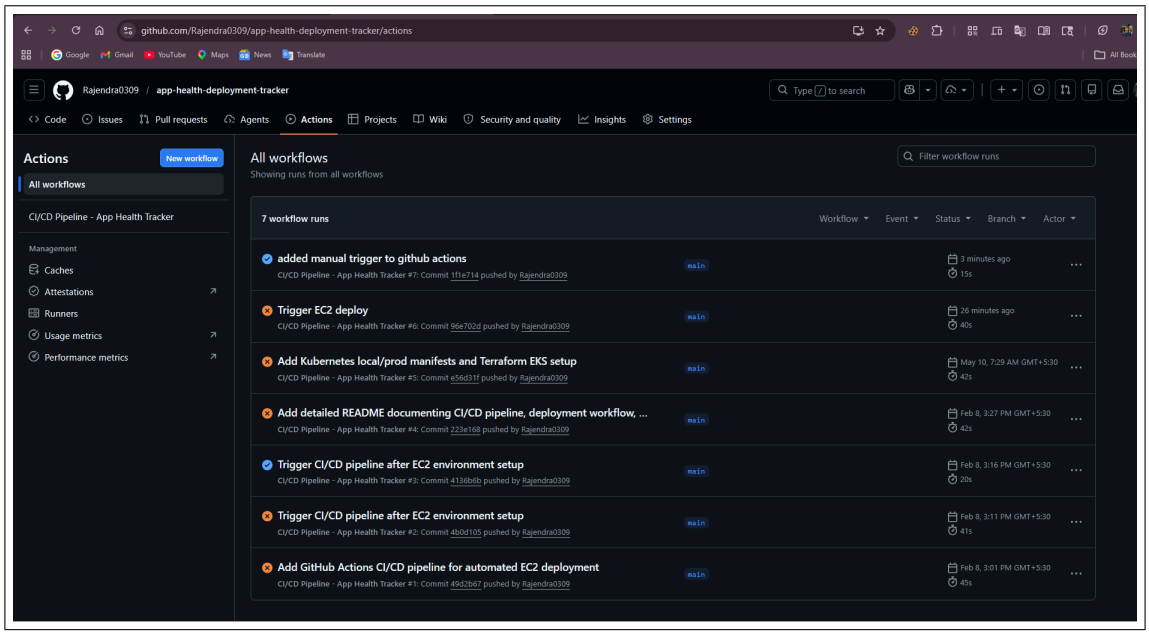


Figure 22: GitHub Actions run

Figure 23: EC2 container running

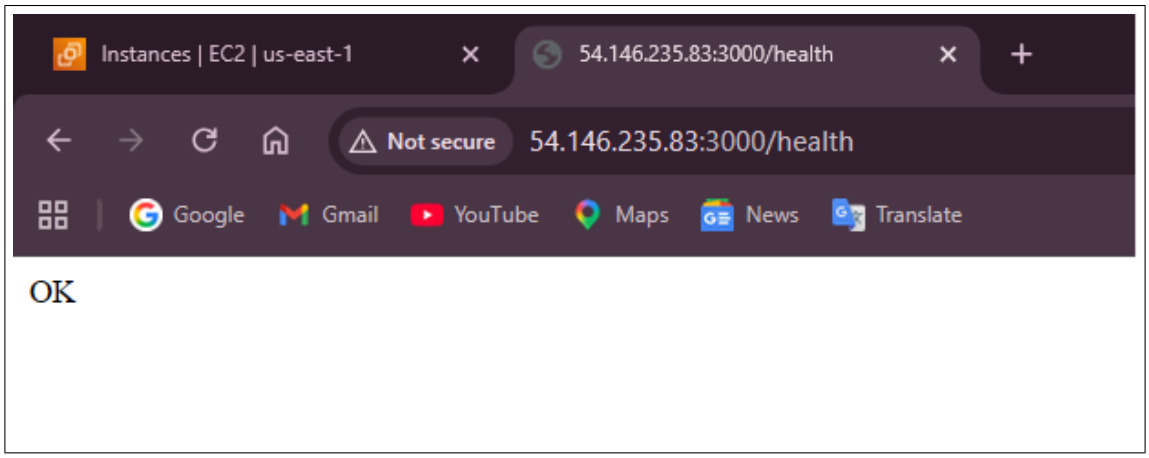


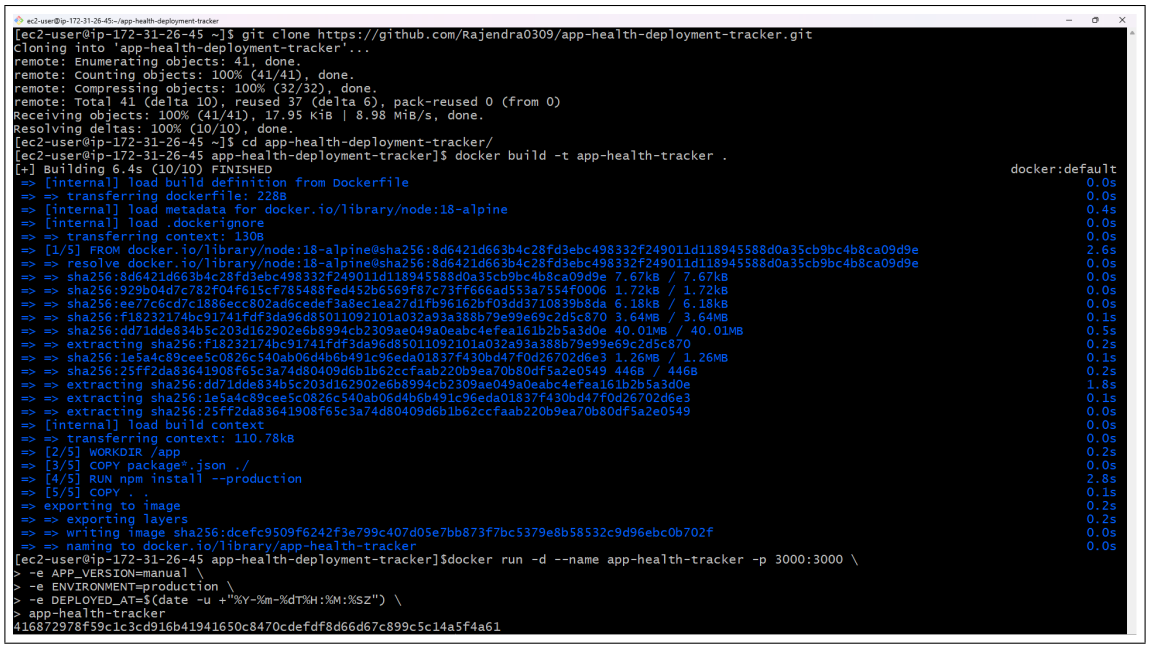
Figure 24: EC2 app response

```
ec2-user@ip-172-31-26-45:~

RAJU@LAPTOP-3G7LJH0J MINGW64 ~/Downloads
$ ssh -i demo-docker.pem ec2-user@54.146.235.83
** WARNING: connection is not using a post-quantum key ex
change algorithm.
** This session may be vulnerable to "store now, decrypt
later" attacks.
** The server may need to be upgraded. See https://openss
h.com/pq.html
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-
linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Wed May 13 05:28:32 2026 from 103.5.133.154
[ec2-user@ip-172-31-26-45 ~]$ sudo yum update -y
Last metadata expiration check: 0:01:51 ago on Wed May 13
 05:29:20 2026.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-26-45 ~]$ sudo yum install git docker
 -y
Last metadata expiration check: 0:02:09 ago on Wed May 13
 05:29:20 2026.
Dependencies resolved.
==========================================================
 Package  Arch   Version                Repository   Size
==========================================================
Installing:
 docker   x86_64 25.0.14-1.amzn2023.0.4
                                        amazonlinux  46 M
 git      x86_64 2.50.1-1.amzn2023.0.1 amazonlinux  53 k
Installing dependencies:
 container-selinux
          noarch 4:2.245.0-1.amzn2023  amazonlinux  58 k
 containerd
          x86_64 2.2.3-1.amzn2023.0.1  amazonlinux  24 M
 git-core x86_64 2.50.1-1.amzn2023.0.1 amazonlinux 4.9 M
```

Figure 25: EC2 SSH session (1)

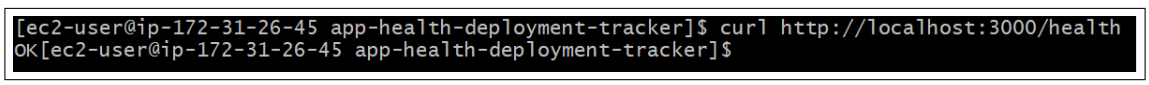
```
Installed:
  container-selinux-4:2.245.0-1.amzn2023.noarch
  containerd-2.2.3-1.amzn2023.0.1.x86_64
  docker-25.0.14-1.amzn2023.0.4.x86_64
  git-2.50.1-1.amzn2023.0.1.x86_64
  git-core-2.50.1-1.amzn2023.0.1.x86_64
  git-core-doc-2.50.1-1.amzn2023.0.1.noarch
  iptables-libs-1.8.8-3.amzn2023.0.2.x86_64
  iptables-nft-1.8.8-3.amzn2023.0.2.x86_64
  libcgroup-3.0-1.amzn2023.0.1.x86_64
  libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64
  libnfnetlink-1.0.1-19.amzn2023.0.2.x86_64
  libnftnl-1.2.2-2.amzn2023.0.2.x86_64
  perl-Error-1:0.17030-2.amzn2023.0.1.noarch
  perl-File-Find-1.37-477.amzn2023.0.7.noarch
  perl-Git-2.50.1-1.amzn2023.0.1.noarch
  perl-TermReadKey-2.38-9.amzn2023.0.2.x86_64
  perl-lib-0.65-477.amzn2023.0.7.x86_64
  pigz-2.5-1.amzn2023.0.3.x86_64
  runc-1.3.4-3.amzn2023.0.2.x86_64

Complete!
[ec2-user@ip-172-31-26-45 ~]$ sudo service docker start
Redirecting to /bin/systemctl start docker.service
[ec2-user@ip-172-31-26-45 ~]$ sudo usermod -aG docker ec2
-user
```

Figure 26: EC2 SSH session (2)



```
ec2-user@ip-172-31-26-45:~/app-health-deployment-tracker
[ec2-user@ip-172-31-26-45 ~]$ git clone https://github.com/Rajendra0309/app-health-deployment-tracker.git
Cloning into 'app-health-deployment-tracker'...
remote: Enumerating objects: 41, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (32/32), done.
remote: Total 41 (delta 10), reused 37 (delta 6), pack-reused 0 (from 0)
Receiving objects: 100% (41/41), 17.95 KiB | 8.98 MiB/s, done.
Resolving deltas: 100% (10/10), done.
[ec2-user@ip-172-31-26-45 ~]$ cd app-health-deployment-tracker/
[ec2-user@ip-172-31-26-45 app-health-deployment-tracker]$ docker build -t app-health-tracker .
[+] Building 6.4s (10/10) FINISHED                                                                          docker:default
 => [internal] load build definition from Dockerfile                                                                   0.0s
 => => transferring dockerfile: 228B                                                                                   0.0s
 => [internal] load metadata for docker.io/library/node:18-alpine                                                      0.4s
 => [internal] load .dockerignore                                                                                      0.0s
 => => transferring context: 130B                                                                                      0.0s
 => [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e  2.6s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e  0.0s
 => => sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e 7.67kB / 7.67kB                         0.0s
 => => sha256:929b04d7c782f04f615cf785488fed452b6569f87c73ff666ad553a7554f0006 1.72kB / 1.72kB                         0.0s
 => => sha256:ee77c6cd7c1886ecc802ad6cedef3a8ec1ea27d1fb96162bf03dd371083 9b8da 6.18kB / 6.18kB                       0.0s
 => => sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB                         0.1s
 => => sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 40.01MB / 40.01MB                       0.5s
 => => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870                              0.2s
 => => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB                         0.1s
 => => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B                             0.2s
 => => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e                              1.8s
 => => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3                              0.1s
 => => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549                              0.0s
 => [internal] load build context                                                                                      0.0s
 => => transferring context: 110.78kB                                                                                  0.0s
 => [2/5] WORKDIR /app                                                                                                 0.2s
 => [3/5] COPY package*.json ./                                                                                        0.0s
 => [4/5] RUN npm install --production                                                                                 2.8s
 => [5/5] COPY . .                                                                                                     0.1s
 => exporting to image                                                                                                 0.2s
 => => exporting layers                                                                                                0.2s
 => => writing image sha256:dcefc9509f6242f3e799c407d05e7bb873f7bc5379e8b58532c9d96ebc0b702f                           0.0s
 => => naming to docker.io/library/app-health-tracker                                                                  0.0s
[ec2-user@ip-172-31-26-45 app-health-deployment-tracker]$docker run -d --name app-health-tracker -p 3000:3000 \
> -e APP_VERSION=manual \
> -e ENVIRONMENT=production \
> -e DEPLOYED_AT=$(date -u +"%Y-%m-%dT%H:%M:%SZ") \
> app-health-tracker
416872978f59c1c3cd916b41941650c8470cdefdf8d66d67c899c5c14a5f4a61
```

Figure 27: EC2 docker run



```
[ec2-user@ip-172-31-26-45 app-health-deployment-tracker]$ curl http://localhost:3000/health
OK[ec2-user@ip-172-31-26-45 app-health-deployment-tracker]$
```

Figure 28: EC2 curl /health

9 Cleanup (Avoid Charges)

9.1 Commands

```
kubectl delete -f k8s\prod\service.yaml
kubectl delete -f k8s\prod\deployment.yaml
kubectl delete -f k8s\prod\configmap.yaml
aws ecr delete-repository --repository-name app-health-tracker --region us-east-1 --force
cd terraform
terraform destroy
```

9.2 Screenshots

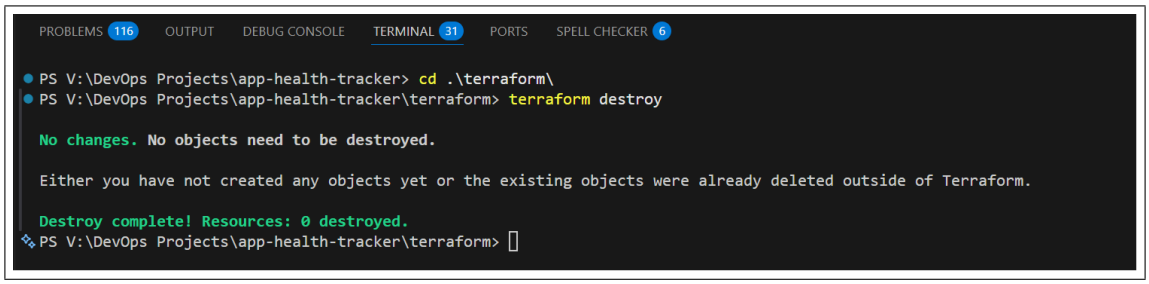


Figure 29: Terraform destroy